

Integrating Planning and Learning: Reinforcement Learning - Lecture 10

Felipe Meneguzzi
`felipe.meneguzzi@pucrs.br`

PUCRS

Material adapted from:
Martha and Adam White - University of Alberta/Coursera
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)
David Silver - Reinforcement Learning

Outline

- 1 Recap
- 2 Introduction
- 3 Model-Based Reinforcement Learning
 - Learning a Model
 - Planning with a Model
- 4 Integrated Architectures
 - Dyna
 - Dyna-Q+
- 5 Simulation-Based Search
 - Monte-Carlo Search
 - MCTS in Go
 - Temporal Difference Search

- Last lecture:
 - n-Step **Bootstrapping** and λ updates
 - Eligibility Traces
- This lecture (Sutton and Barto 8.1-8.4)
 - **Integrating Planning and Learning**
 - Dyna-Q
- Next lecture
 - Function approximations
 - Approximation-based prediction and Control

Recap

- We studied how to compute value-functions (i.e., to plan)

Model-Free by learning via Temporal-Difference and Monte-Carlo updates
and interpolate between these two updates via n -bootstrapping and λ -updates

Model-Based by direct computation via dynamic programming

- Can we also interpolate between model-free and model-based planning?

Outline

- 1 Recap
- 2 Introduction
- 3 Model-Based Reinforcement Learning
 - Learning a Model
 - Planning with a Model
- 4 Integrated Architectures
 - Dyna
 - Dyna-Q+
- 5 Simulation-Based Search
 - Monte-Carlo Search
 - MCTS in Go
 - Temporal Difference Search

Model-Based Reinforcement Learning

- Previous lectures: learn **value function** directly from experience
- This lecture: learn **model** directly from experience and use **planning** to construct a value function or policy
- Integrate learning and planning into a single architecture

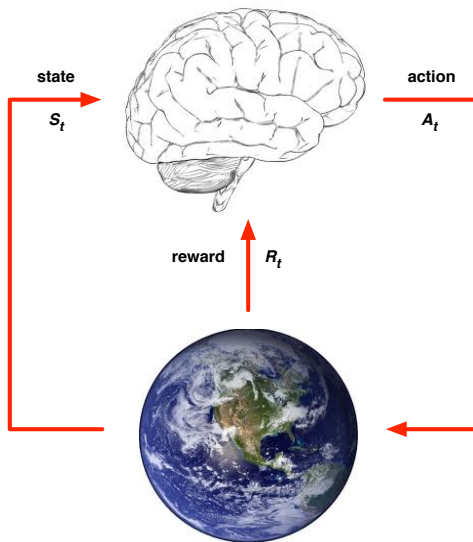
Model-Based and Model-Free RL

- Model-Free RL
 - No model
 - **Learn** value function (and/or policy) from experience

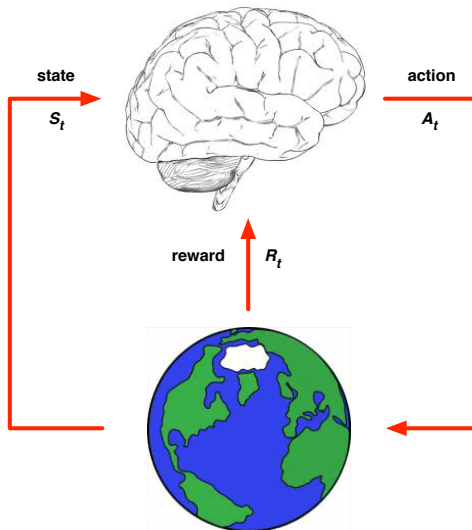
Model-Based and Model-Free RL

- Model-Free RL
 - No model
 - **Learn** value function (and/or policy) from experience
- Model-Based RL
 - Learn a model from experience
 - **Plan** value function (and/or policy) from model

Model-Free RL



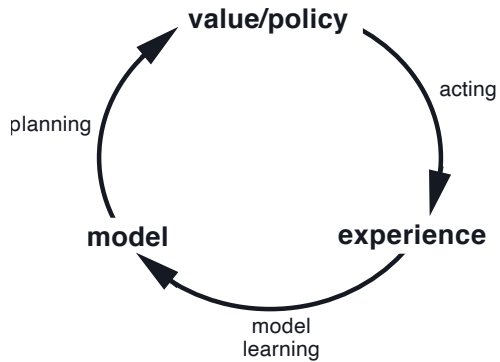
Model-Based RL



Outline

- 1 Recap
- 2 Introduction
- 3 Model-Based Reinforcement Learning
 - Learning a Model
 - Planning with a Model
- 4 Integrated Architectures
 - Dyna
 - Dyna-Q+
- 5 Simulation-Based Search
 - Monte-Carlo Search
 - MCTS in Go
 - Temporal Difference Search

Model-Based RL



Advantages of Model-Based RL

Advantages:

- Can efficiently learn model by supervised learning methods
- Can reason about model uncertainty

Disadvantages:

- First learn a model, then construct a value function
 - two sources of approximation error

What is a Model?

- A **model** $\mathcal{M}$ is a representation of an MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$, parameterized by η
- We will assume state space $\mathcal{S}$ and action space $\mathcal{A}$ are known
- So a model $\mathcal{M}_\eta = \langle \mathcal{P}_\eta, \mathcal{R}_\eta \rangle$ represents state transitions $\mathcal{P}_\eta \approx \mathcal{P}$ and rewards $\mathcal{R}_\eta \approx \mathcal{R}$

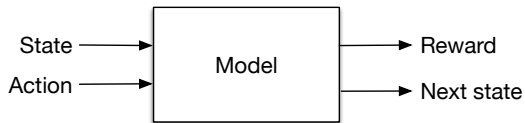
$$S_{t+1} \sim \mathcal{P}_\eta(S_{t+1} \mid S_t, A_t)$$

$$R_{t+1} = \mathcal{R}_\eta(R_{t+1} \mid S_t, A_t)$$

- Typically assume conditional independence between state transitions and rewards

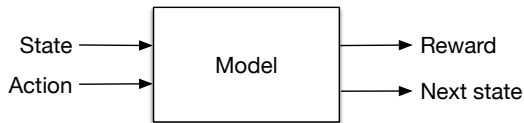
$$\mathbb{P}[S_{t+1}, R_{t+1} \mid S_t, R_t] = \mathbb{P}[S_{t+1} \mid S_t, R_t] \mathbb{P}[R_{t+1} \mid S_t, R_t]$$

What is a Model?



- **Sample Models** are akin to simulator **blackbox**
 - Capable of generating one-step predictions in state-space
 - Low computational cost, low information value
e.g. single die to generate 12-dice rolls

What is a Model?



- **Sample Models** are akin to simulator **blackbox**
 - Capable of generating one-step predictions in state-space
 - Low computational cost, low information value
e.g. single die to generate 12-dice rolls
- **Distribution Models** are complete **mathematical models** for the state-space/action-space
 - Capable of providing the complete **probability distributions** for all events
 - High computational cost, high information value
e.g. the transition model $\mathbb{P}[s', r \mid s, a]$ in an MDP

Model Learning

- Goal: estimate model $\mathcal{M}_\eta$ from experience $\{S_1, A_1, R_2, \dots, S_T\}$
- This is a supervised learning problem:

$$\begin{aligned} S_1, A_1 &\rightarrow R_2, S_2 \\ S_2, A_2 &\rightarrow R_3, S_3 \\ &\vdots \\ S_{T-1}, A_{T-1} &\rightarrow R_T, S_T \end{aligned}$$

- Learning $s, a \rightarrow r$ is a **regression** problem
- Learning $s, a \rightarrow s'$ is a **density estimation** problem
- Pick loss function, e.g. mean-squared error, KL divergence, ...
- Find parameters η that minimize empirical loss

Examples of Models

- Table Lookup Model
- Linear Expectation Model
- Linear Gaussian Model
- Gaussian Process Model
- Deep Belief Network Model
- ...

Table Lookup Model

- Model is an explicit MDP, $\hat{\mathcal{P}}, \hat{\mathcal{R}}$
- Count visits $N(s, a)$ to each state action pair

$$\hat{\mathcal{P}}_{ss'}^a = \frac{1}{N(s, a)} \sum_{t=1}^T 1(S_t, A_t, S_{t+1} = s, a, s')$$

$$\hat{\mathcal{R}}_s^a = \frac{1}{N(s, a)} \sum_{t=1}^T 1(S_t, A_t = s, a) R_t$$

- Alternatively
 - At each time step t , record experience tuple $\langle S_t, A_t, R_{t+1}, S_{t+1} \rangle$
 - To sample model, randomly pick tuple matching $\langle s, a, \cdot, \cdot \rangle$

AB Example

Two states A , B ; no discounting; 8 episodes of experience

$A, 0, B, 0$

$B, 1$

$B, 1$

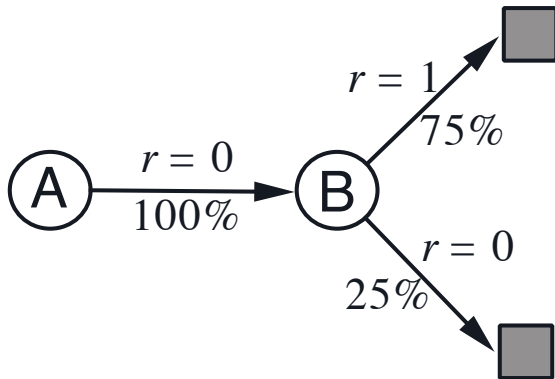
$B, 1$

$B, 1$

$B, 1$

$B, 1$

$B, 0$



We have constructed a **table lookup model** from the experience

Planning with a Model

- Given a model $\mathcal{M}_\eta = \langle \mathcal{P}_\eta, \mathcal{R}_\eta \rangle$
- Solve the MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}_\eta, \mathcal{R}_\eta \rangle$
- Using favorite planning algorithm
 - Value iteration
 - Policy iteration
 - Tree search
 - ...

Sample-Based Planning

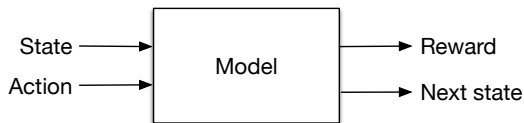
- A simple but powerful approach to planning
- Use the model **only** to generate samples
- **Sample** experience from model

$$S_{t+1} \sim \mathcal{P}_\eta(S_{t+1} \mid S_t, A_t)$$

$$R_{t+1} = \mathcal{P}_\eta(R_{t+1} \mid S_t, A_t)$$

- Apply **model-free** RL to samples, e.g.:
 - Monte-Carlo control
 - Sarsa
 - Q-learning
- Sample-based planning methods are often more efficient

Random Tabular Q-Planning



Random-sample one-step tabular Q-planning

- 1: **loop** forever
- 2: Select a state, $S \in \mathcal{S}$, and an action, $A \in \mathcal{A}(S)$, at random
- 3: $R, S' \leftarrow \text{Model}(S, A)$
- 4: $\triangleright$ Apply one-step tabular Q-learning to S, A, R, S'
- 5: $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

Back to the AB Example

- Construct a table-lookup model from real experience
- Apply model-free RL to sampled experience

Real experience

A, 0, B, 0

B, 1

B, 1

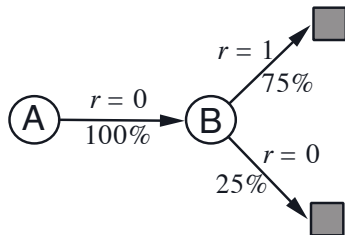
B, 1

B, 1

B, 1

B, 1

B, 0



Sampled experience

B, 0

B, 0

B, 1

A, 0, B, 1

B, 1

A, 0, B, 1

B, 1

B, 0

e.g. Monte-Carlo learning: $V(A) = 1$, $V(B) = 0.75$

Planning with an Inaccurate Model

- Given an imperfect model $\langle \mathcal{P}_\eta, \mathcal{R}_\eta \rangle \neq \langle \mathcal{P}, \mathcal{R} \rangle$
- Performance of model-based RL is limited to optimal policy for approximate MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}_\eta, \mathcal{R}_\eta \rangle$
- i.e. Model-based RL is only as good as the estimated model
- When the model is inaccurate, planning process will compute a suboptimal policy
 - Solution 1: when model is wrong, use model-free RL
 - Solution 2: reason explicitly about model uncertainty

Outline

- 1 Recap
- 2 Introduction
- 3 Model-Based Reinforcement Learning
 - Learning a Model
 - Planning with a Model
- 4 Integrated Architectures
 - Dyna
 - Dyna-Q+
- 5 Simulation-Based Search
 - Monte-Carlo Search
 - MCTS in Go
 - Temporal Difference Search

Real and Simulated Experience

We consider two sources of experience

Real experience Sampled from environment (true MDP)

$$S' \sim \mathcal{P}_{s,s'}^a$$

$$R = \mathcal{R}_s^a$$

Simulated experience Sampled from model (approximate MDP)

$$S' \sim \mathcal{P}_\eta(S' \mid S, R)$$

$$R = \mathcal{R}_\eta(\mathcal{R} \mid S, R)$$

Integrating Learning and Planning

- Model-Free RL
 - No model
 - **Learn** value function (and/or policy) from real experience

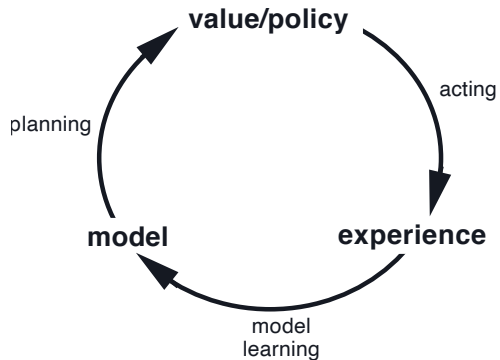
Integrating Learning and Planning

- Model-Free RL
 - No model
 - **Learn** value function (and/or policy) from real experience
- Model-Based RL (using Sample-Based Planning)
 - Learn a model from real experience
 - **Plan** value function (and/or policy) from simulated experience

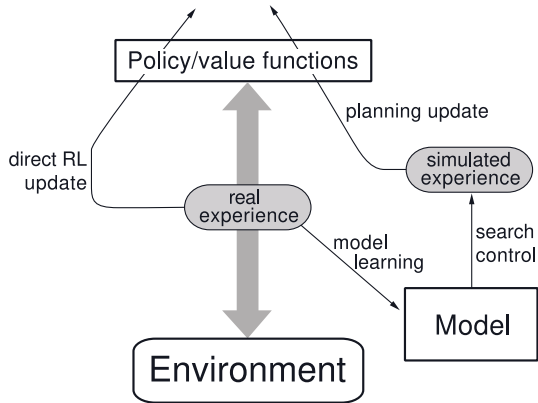
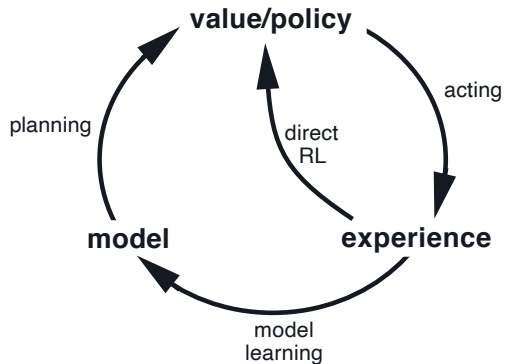
Integrating Learning and Planning

- Model-Free RL
 - No model
 - **Learn** value function (and/or policy) from real experience
- Model-Based RL (using Sample-Based Planning)
 - Learn a model from real experience
 - **Plan** value function (and/or policy) from simulated experience
- Dyna
 - Learn a model from real experience
 - **Learn and plan** value function (and/or policy) from real and simulated experience

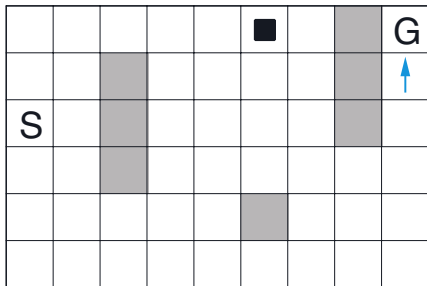
The Dyna Architecture



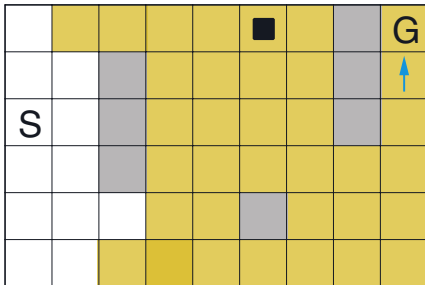
The Dyna Architecture



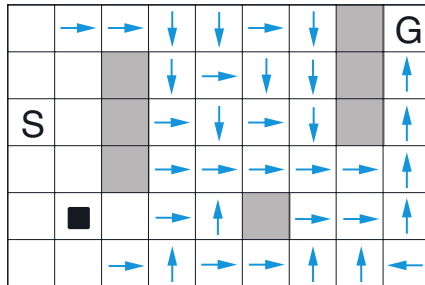
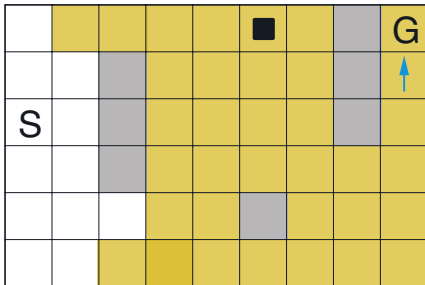
The Dyna Architecture



The Dyna Architecture



The Dyna Architecture



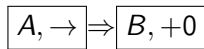
The Dyna Algorithm

Model

A	B
C	D

The Dyna Algorithm

Model



A	B
C	D

The Dyna Algorithm

Model

$A, \rightarrow$	$\Rightarrow$	$B, +0$
$B, \rightarrow$	$\Rightarrow$	$B, +1$

A	$\textcircled{B}$
C	D

The Dyna Algorithm

Model

$A, \rightarrow$	$\Rightarrow$	$B, +0$
$B, \rightarrow$	$\Rightarrow$	$B, +1$
$B, \leftarrow$	$\Rightarrow$	$A, +0$

A	B
C	D

Dyna-Q Algorithm

Tabular Dyna-Q

- 1: **function** dynaQ(α, ϵ, n)
- 2: Initialize $Q(s, a)$ and $\text{model}(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
- 3: **loop** forever
- 4: (a) $S \leftarrow$ current (nonterminal) state
- 5: (b) $A \leftarrow \epsilon\text{-greedy}(S, Q)$
- 6: (c) Execute action A ; observe resultant reward R , and state S'
- 7: (d) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

Dyna-Q Algorithm

Tabular Dyna-Q

- 1: **function** dynaQ(α, ϵ, n)
- 2: Initialize $Q(s, a)$ and $\text{model}(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
- 3: **loop** forever
- 4: (a) $S \leftarrow$ current (nonterminal) state
- 5: (b) $A \leftarrow \epsilon\text{-greedy}(S, Q)$
- 6: (c) Execute action A ; observe resultant reward R , and state S'
- 7: (d) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
- 8: (e) $\text{Model}(S, A) \leftarrow R, S'$

Dyna-Q Algorithm

Tabular Dyna-Q

```
1: function dynaQ( $\alpha, \epsilon, n$ )
2:   Initialize  $Q(s, a)$  and  $\text{model}(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ 
3:   loop forever
4:     (a)  $S \leftarrow$  current (nonterminal) state
5:     (b)  $A \leftarrow \epsilon\text{-greedy}(S, Q)$ 
6:     (c) Execute action  $A$ ; observe resultant reward  $R$ , and state  $S'$ 
7:     (d)  $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
8:     (e)  $\text{Model}(S, A) \leftarrow R, S'$ 

10:     $S \leftarrow$  random previously observed state
11:     $A \leftarrow$  random action previously taken in  $S$ 
12:     $R, S' \leftarrow \text{Model}(S, A)$ 
```

Dyna-Q Algorithm

Tabular Dyna-Q

```
1: function dynaQ( $\alpha, \epsilon, n$ )
2:   Initialize  $Q(s, a)$  and  $\text{model}(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ 
3:   loop forever
4:     (a)  $S \leftarrow$  current (nonterminal) state
5:     (b)  $A \leftarrow \epsilon\text{-greedy}(S, Q)$ 
6:     (c) Execute action  $A$ ; observe resultant reward  $R$ , and state  $S'$ 
7:     (d)  $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
8:     (e)  $\text{Model}(S, A) \leftarrow R, S'$ 

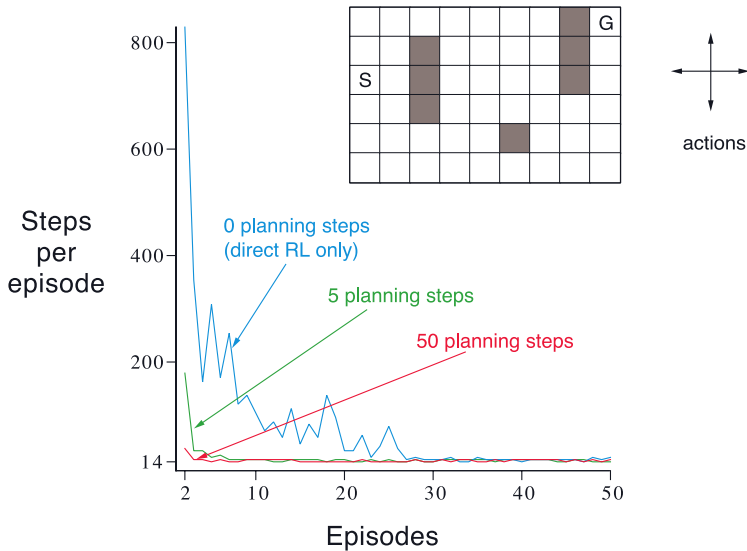
10:     $S \leftarrow$  random previously observed state
11:     $A \leftarrow$  random action previously taken in  $S$ 
12:     $R, S' \leftarrow \text{Model}(S, A)$ 
13:     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
```

Dyna-Q Algorithm

Tabular Dyna-Q

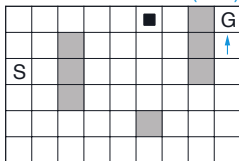
```
1: function dynaQ( $\alpha, \epsilon, n$ )
2:   Initialize  $Q(s, a)$  and  $\text{model}(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ 
3:   loop forever
4:     (a)  $S \leftarrow$  current (nonterminal) state
5:     (b)  $A \leftarrow \epsilon\text{-greedy}(S, Q)$ 
6:     (c) Execute action  $A$ ; observe resultant reward  $R$ , and state  $S'$ 
7:     (d)  $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
8:     (e)  $\text{Model}(S, A) \leftarrow R, S'$ 
9:     loop repeat  $n$  times
10:       $S \leftarrow$  random previously observed state
11:       $A \leftarrow$  random action previously taken in  $S$ 
12:       $R, S' \leftarrow \text{Model}(S, A)$ 
13:       $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
```

Dyna-Q on a Simple Maze

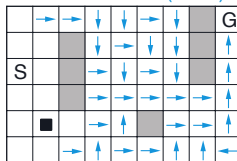


Dyna-Q on a Simple Maze

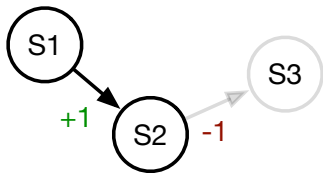
WITHOUT PLANNING ($n=0$)



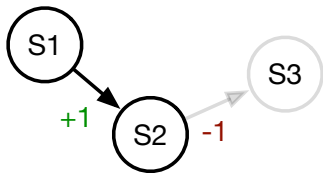
WITH PLANNING ($n=50$)



What if the model is Wrong?



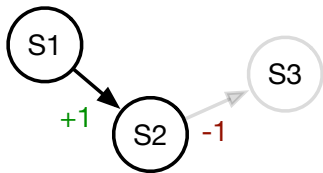
What if the model is Wrong?



S1 $\rightarrow$ S2, +1
S2 $\rightarrow$??

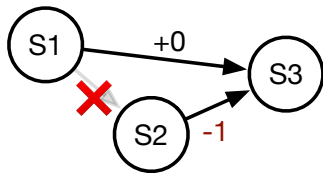
Incomplete Model

What if the model is Wrong?

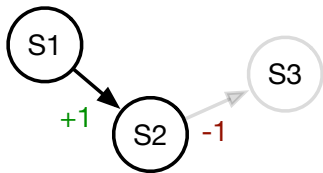


S1 $\rightarrow$ S2, +1
S2 $\rightarrow$??

Incomplete Model

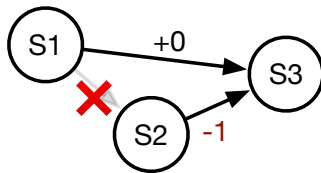


What if the model is Wrong?



S1 → S2, +1
S2 → ??

Incomplete Model



S1 → S2, +1 ~~X~~
S2 → S3, -1

Changing Environment

Exploration and Exploitation for model accuracy

- A changing environment creates another exploration vs. exploitation dilemma:
 - Exploit the existing model in the planning step?
 - (Re)explore the environment in the simulation step to update the model?

Exploration and Exploitation for model accuracy

- A changing environment creates another exploration vs. exploitation dilemma:
 - Exploit the existing model in the planning step?
 - (Re)explore the environment in the simulation step to update the model?
- Possible update principle: as model gets old, it becomes less reliable

Bonus rewards for exploration / Dyna-Q+

$$r \leftarrow \underbrace{r} + \underbrace{\kappa}_{\sqrt{\tau}}$$

Bonus rewards for exploration / Dyna-Q+

$$r \leftarrow \underbrace{r}_{\text{Actual Reward}} + \underbrace{\kappa}_{\text{Exploration Bonus}} \underbrace{\sqrt{\tau}}_{\text{Exploration Bonus}}$$

Bonus rewards for exploration / Dyna-Q+

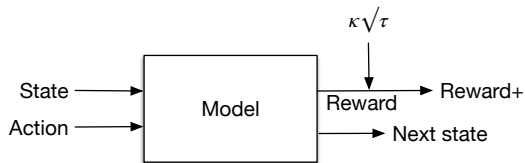
$$r \leftarrow \underbrace{r}_{\text{Actual Reward}} + \underbrace{\kappa}_{\text{Constant}} \sqrt{\tau}$$

Bonus rewards for exploration / Dyna-Q+

$$r \leftarrow \underbrace{r}_{\text{Actual Reward}} + \underbrace{\kappa}_{\text{Constant}} \underbrace{\sqrt{\tau}}_{\text{TimeSteps}}$$

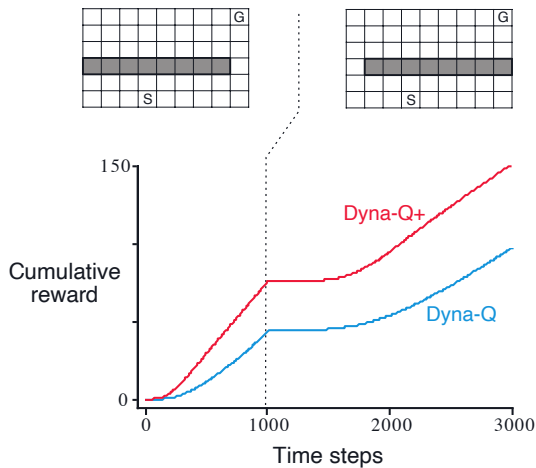
Bonus rewards for exploration / Dyna-Q+

$$r \leftarrow \underbrace{r}_{\text{Actual Reward}} + \underbrace{\kappa}_{\text{Constant}} \underbrace{\sqrt{\tau}}_{\text{TimeSteps}}$$



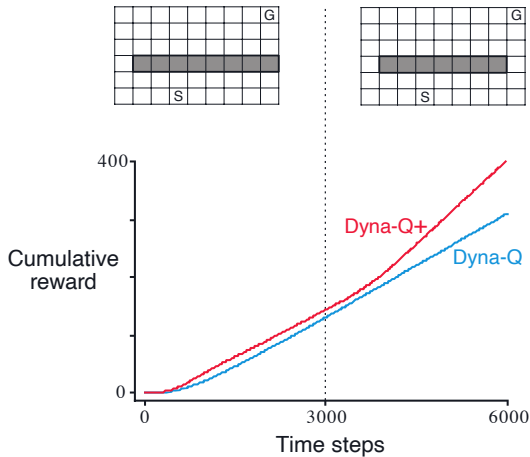
Dyna-Q with an Inaccurate Model

- The changed environment is **harder**



Dyna-Q with an Inaccurate Model (2)

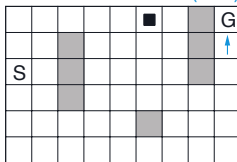
- The changed environment is **easier**



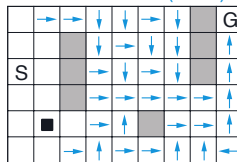
Prioritized Sweeping: Dyna-Q on a Simple Maze

Why do we need so many planning steps?

WITHOUT PLANNING ($n=0$)



WITH PLANNING ($n=50$)



Prioritized Sweeping

- In much larger state-spaces, unfocused search is extremely inefficient
- States close to where changes occurred are more likely to change in future iterations
- Thus, we can speed up planning by **backward focusing** planning computations
- This is the idea behind **prioritized sweeping**

Prioritized Sweeping Algorithm

Prioritized sweeping for deterministic environment

- 1: **function** prioritizedDynaQ($\alpha, \epsilon, n, \theta$)
- 2: Initialize $Q(s, a)$ and $\text{model}(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $PQueue$ empty
- 3: **loop** forever
- 4: (a) $S \leftarrow$ current (nonterminal) state
- 5: (b) $A \leftarrow \text{policy}(S, Q)$
- 6: (c) Execute action A ; observe resultant reward R , and state S'
- 7: (d) $\text{Model}(S, A) \leftarrow R, S'$

Prioritized Sweeping Algorithm

Prioritized sweeping for deterministic environment

```
1: function prioritizedDynaQ( $\alpha, \epsilon, n, \theta$ )
2:   Initialize  $Q(s, a)$  and  $\text{model}(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ , and  $PQueue$  empty
3:   loop forever
4:     (a)  $S \leftarrow$  current (nonterminal) state
5:     (b)  $A \leftarrow \text{policy}(S, Q)$ 
6:     (c) Execute action  $A$ ; observe resultant reward  $R$ , and state  $S'$ 
7:     (d)  $\text{Model}(S, A) \leftarrow R, S'$ 
8:     (e)  $P \leftarrow |R + \gamma \max_a Q(S', a) - Q(S, A)|$ 
```

Prioritized Sweeping Algorithm

Prioritized sweeping for deterministic environment

```
1: function prioritizedDynaQ( $\alpha, \epsilon, n, \theta$ )
2:   Initialize  $Q(s, a)$  and  $\text{model}(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ , and  $PQueue$  empty
3:   loop forever
4:     (a)  $S \leftarrow$  current (nonterminal) state
5:     (b)  $A \leftarrow \text{policy}(S, Q)$ 
6:     (c) Execute action  $A$ ; observe resultant reward  $R$ , and state  $S'$ 
7:     (d)  $\text{Model}(S, A) \leftarrow R, S'$ 
8:     (e)  $P \leftarrow |R + \gamma \max_a Q(S', a) - Q(S, A)|$ 
9:     (f) if  $P > \theta$ , then insert  $S, A$  into  $PQueue$  with priority  $P$ 
```


Prioritized Sweeping Algorithm

Prioritized sweeping for deterministic environment

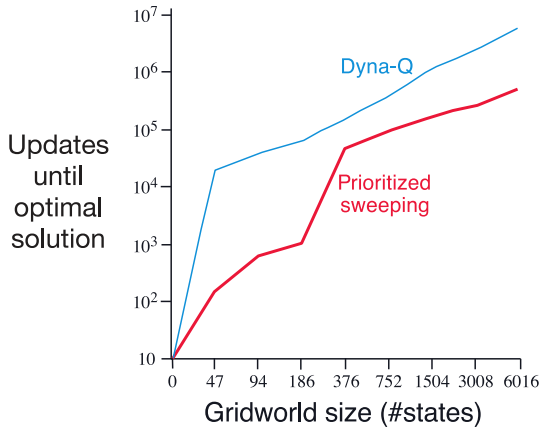
```
1: function prioritizedDynaQ( $\alpha, \epsilon, n, \theta$ )
2:   Initialize  $Q(s, a)$  and  $\text{model}(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ , and  $PQueue$  empty
3:   loop forever
4:     (a)  $S \leftarrow$  current (nonterminal) state
5:     (b)  $A \leftarrow \text{policy}(S, Q)$ 
6:     (c) Execute action  $A$ ; observe resultant reward  $R$ , and state  $S'$ 
7:     (d)  $\text{Model}(S, A) \leftarrow R, S'$ 
8:     (e)  $P \leftarrow |R + \gamma \max_a Q(S', a) - Q(S, A)|$ 
9:     (f) if  $P > \theta$ , then insert  $S, A$  into  $PQueue$  with priority  $P$ 
10:    loop repeat  $n$  times, while  $PQueue$  is not empty
11:       $S, A \leftarrow \text{first}(PQueue)$ 
12:       $R, S' \leftarrow \text{Model}(S, A)$ 
13:       $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
```

Prioritized Sweeping Algorithm

Prioritized sweeping for deterministic environment

```
1: function prioritizedDynaQ( $\alpha, \epsilon, n, \theta$ )
2:   Initialize  $Q(s, a)$  and  $\text{model}(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ , and  $PQueue$  empty
3:   loop forever
4:     (a)  $S \leftarrow$  current (nonterminal) state
5:     (b)  $A \leftarrow \text{policy}(S, Q)$ 
6:     (c) Execute action  $A$ ; observe resultant reward  $R$ , and state  $S'$ 
7:     (d)  $\text{Model}(S, A) \leftarrow R, S'$ 
8:     (e)  $P \leftarrow |R + \gamma \max_a Q(S', a) - Q(S, A)|$ 
9:     (f) if  $P > \theta$ , then insert  $S, A$  into  $PQueue$  with priority  $P$ 
10:    loop repeat  $n$  times, while  $PQueue$  is not empty
11:       $S, A \leftarrow \text{first}(PQueue)$ 
12:       $R, S' \leftarrow \text{Model}(S, A)$ 
13:       $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
14:      loop for all  $\bar{S}, \bar{A}$  predicted to lead to  $S$ 
15:         $\bar{R} \leftarrow$  predicted reward for  $\bar{S}, \bar{A}, S$ 
16:         $P \leftarrow |\bar{R} + \gamma \max_a Q(S, a) - Q(\bar{S}, \bar{A})|$ 
17:        if  $P > \theta$ , then insert  $\bar{S}, \bar{A}$  into  $PQueue$  with priority  $P$ 
```

Prioritized Sweeping Results

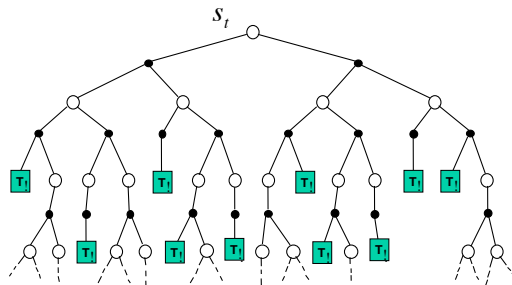


Outline

- 1 Recap
- 2 Introduction
- 3 Model-Based Reinforcement Learning
 - Learning a Model
 - Planning with a Model
- 4 Integrated Architectures
 - Dyna
 - Dyna-Q+
- 5 Simulation-Based Search
 - Monte-Carlo Search
 - MCTS in Go
 - Temporal Difference Search

Forward Search

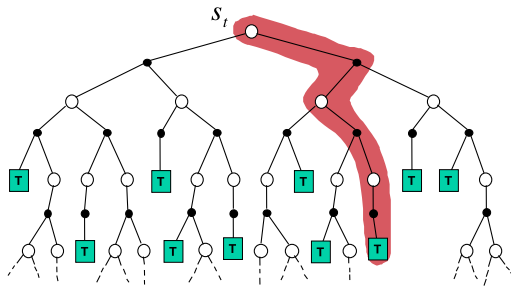
- **Forward search** algorithms select the best action by **lookahead**
- They build a **search tree** with the current state s_t at the root
- Using a **model** of the MDP to look ahead



- No need to solve whole MDP, just sub-MDP starting from **now**

Simulation-Based Search

- **Forward search** paradigm using sample-based planning
- **Simulate** episodes of experience from **now** with the model
- Apply **model-free** RL to simulated episodes



Simulation-Based Search

- Simulate episodes of experience from now with the model

$$\left\{ \textcolor{red}{s}_t^k, A_t^k, R_{t+1}^k, \dots, S_T^k \right\}_{k=1}^K \sim \mathcal{M}_\nu$$

- Apply **model-free** RL to simulated episodes
 - Monte-Carlo control $\rightarrow$ Monte-Carlo search
 - Sarsa $\rightarrow$ TD search
 - ...

Simple Monte-Carlo Search

- Given a model $\mathcal{M}_\nu$ and a **simulation policy** π
- For each action $a \in \mathcal{A}$
 - Simulate K episodes from current (real) state s_t

$$\left\{ \mathbf{s}_t^k, a, R_{t+1}^k, S_{t+1}^k, A_{t+1}^k, \dots, S_T^k \right\}_{k=1}^K \sim \mathcal{M}_\nu, \pi$$

- Evaluate actions by mean return (**Monte-Carlo evaluation**)

$$Q(\mathbf{s}_t, a) = \frac{1}{K} \sum_{k=1}^K G_t \xrightarrow{P} q_\pi(s_t, a)$$

- Select current (real) action with maximum value

$$s_t = \arg \max_{a \in \mathcal{A}} Q(s_t, a)$$

Monte-Carlo Tree Search (Evaluation)

- Given a model $\mathcal{M}_\nu$
- Simulate K episodes from current state s_t using current simulation policy π

$$\left\{ \mathbf{s}_t^k, A_t^k, R_{t+1}^k, S_{t+1}^k, \dots, S_T^k \right\}_{k=1}^K \sim \mathcal{M}_\nu, \pi$$

- Build a search tree containing visited states and actions
- **Evaluate** states $Q(s, a)$ by mean return of episodes from s, a

$$Q(\mathbf{s}, \mathbf{a}) = \frac{1}{N(\mathbf{s}, \mathbf{a})} \sum_{k=1}^K \sum_{u=t}^T \mathbf{1}(S_u, A_u = \mathbf{s}, \mathbf{a}) G_u \xrightarrow{P} q_\pi(\mathbf{s}, \mathbf{a})$$

- After search is finished, select current (real) action with maximum value in search tree

$$\mathbf{s}_t = \arg \max_{\mathbf{a} \in \mathcal{A}} Q(\mathbf{s}_t, \mathbf{a})$$

Monte-Carlo Tree Search (Simulation)

- In MCTS, the simulation policy π **improves**
- Each simulation consists of two phases (in-tree, out-of-tree)
 - **Tree policy** (improves): pick actions to maximize $Q(S, A)$
 - **Default policy** (fixed): pick actions randomly
- Repeat (each simulation)
 - **Evaluate** states $Q(S, A)$ by Monte-Carlo evaluation
 - **Improve** tree policy, e.g. by ϵ -greedy(Q)
- **Monte-Carlo control** applied to **simulated experience**
- Converges on the optimal search tree, $Q(S, A) \rightarrow q_*(S, A)$

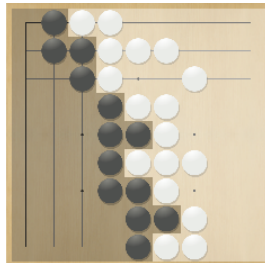
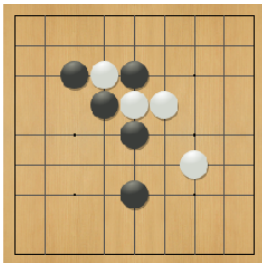
Case Study: the Game of Go

- The ancient oriental game of Go is 2500 years old
- Considered to be the hardest classic board game
- Considered a grand challenge task for AI (John McCarthy)
- Traditional game-tree search has failed in Go



Rules of Go

- Usually played on 19x19, also 13x13 or 9x9 board
- Simple rules, complex strategy
- Black and white place down stones alternately
- Surrounded stones are captured and removed
- The player with more territory wins the game



Position Evaluation in Go

- How good is a position s ?
- Reward function (undiscounted):

$$R_t = 0 \text{ for all non-terminal steps } t < T$$

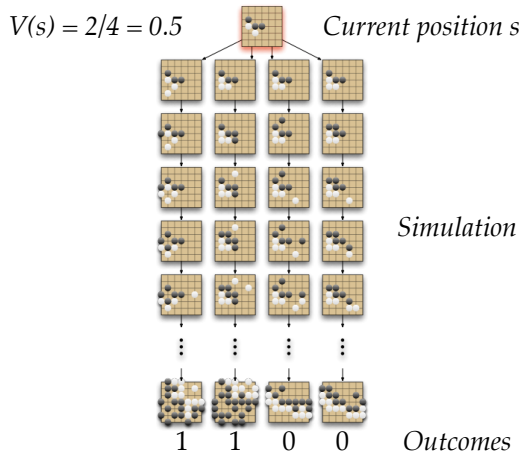
$$R_T = \begin{cases} 1 & \text{if Black wins} \\ 0 & \text{if White wins} \end{cases}$$

- Policy $\pi = \langle \pi_B, \pi_W \rangle$ selects moves for both players
- Value function (how good is position s):

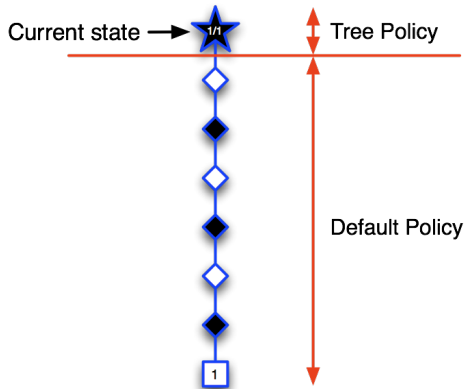
$$v_\pi(s) = \mathbb{E}[R_T \mid S = s] = \mathbb{P}[\text{Black wins} \mid S = s]$$

$$v_*(s) = \max_{\pi_B} \min_{\pi_W} v_\pi(s)$$

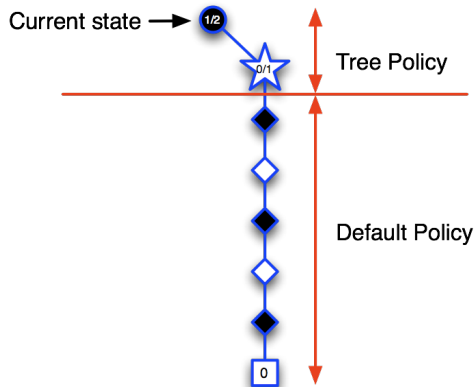
Monte-Carlo Evaluation in Go



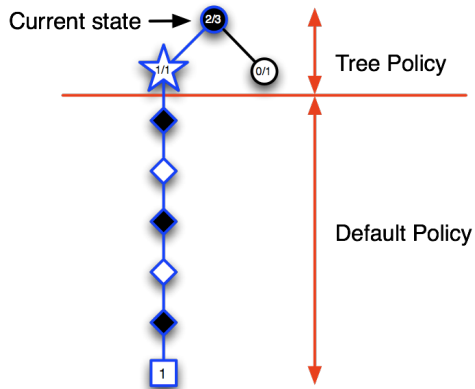
Applying Monte-Carlo Tree Search (1)



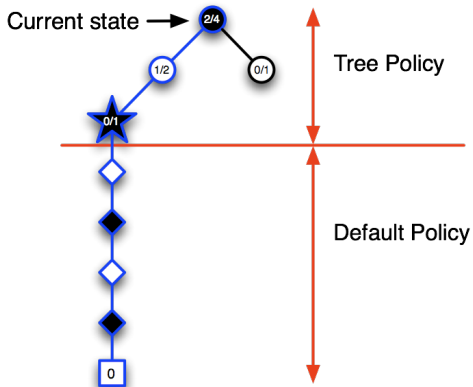
Applying Monte-Carlo Tree Search (2)



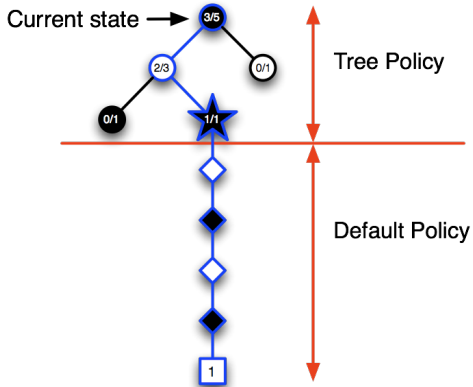
Applying Monte-Carlo Tree Search (3)



Applying Monte-Carlo Tree Search (4)



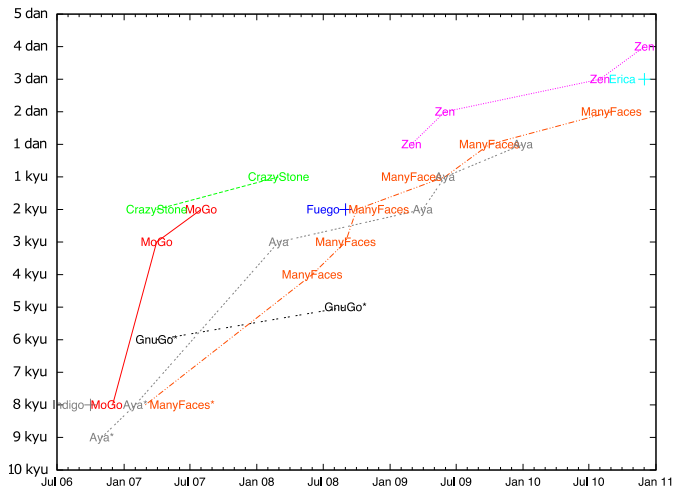
Applying Monte-Carlo Tree Search (5)



Advantages of MC Tree Search

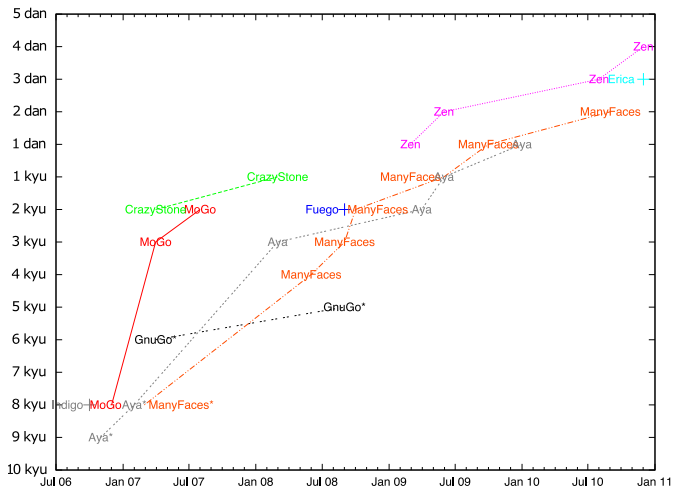
- Highly selective best-first search
- Evaluates states **dynamically** (unlike e.g. DP)
- Uses sampling to break curse of dimensionality
- Works for “black-box models” (only requires samples)
- Computationally efficient, anytime, parallelizable

Example: MC Tree Search in Computer Go



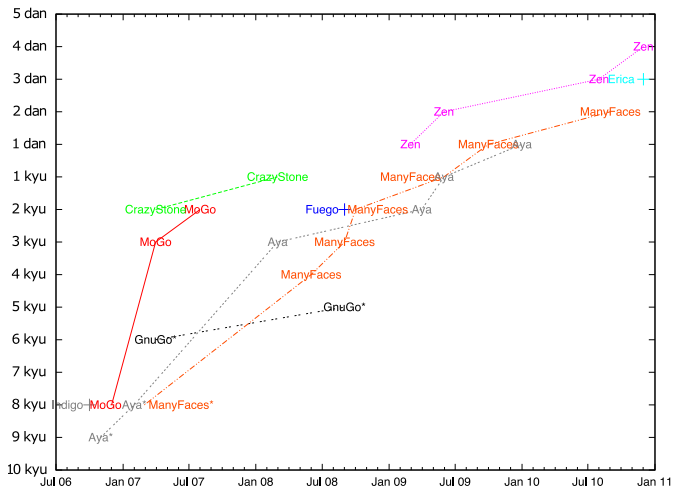
Example: MC Tree Search in Computer Go

- 2015 - AlphaGo - MCTS plus DNN to store policy (CPU+1GPU)



Example: MC Tree Search in Computer Go

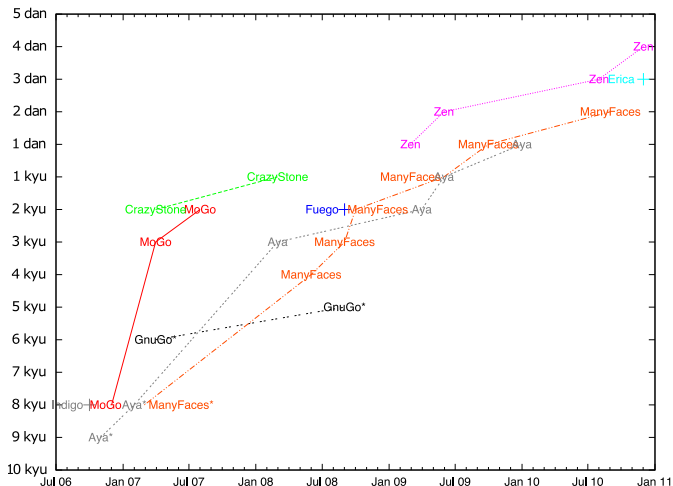
- 2015 - AlphaGo - MCTS plus DNN to store policy (CPU+1GPU)
- 2017 - AlphaGo Zero - MCTS plus DNN to store policy (4 TPUs)



Example: MC Tree Search in Computer Go

- 2015 - AlphaGo - MCTS plus DNN to store policy (CPU+1GPU)
- 2017 - AlphaGo Zero - MCTS plus DNN to store policy (4 TPUs)
- 2019 - MuZero - MCTS plus multiple DNNs to store policy and learned model (1000 TPUs)

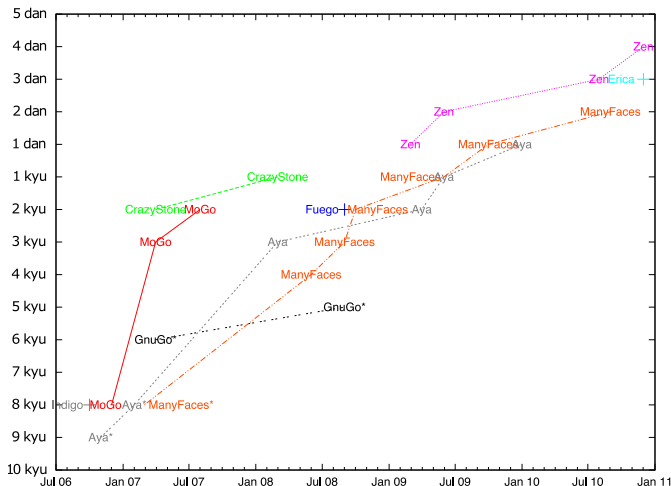
All playing the equivalent of 9 dan



Example: MC Tree Search in Computer Go

- 2015 - AlphaGo - MCTS plus DNN to store policy (CPU+1GPU)
- 2017 - AlphaGo Zero - MCTS plus DNN to store policy (4 TPUs)
- 2019 - MuZero - MCTS plus multiple DNNs to store policy and learned model (1000 TPUs)

All playing the equivalent of 9 dan
How we use DNNs and other ML models in RL (Next Lecture)



Temporal-Difference Search

- Simulation-based search
- Using TD instead of MC (bootstrapping)
- MC tree search applies MC control to sub-MDP from now
- TD search applies Sarsa to sub-MDP from now

MC vs. TD search

- For model-free reinforcement learning, bootstrapping is helpful
 - TD learning reduces variance but increases bias
 - TD learning is usually more efficient than MC
 - $TD(\lambda)$ can be much more efficient than MC
- For simulation-based search, bootstrapping is also helpful
 - TD search reduces variance but increases bias
 - TD search is usually more efficient than MC search
 - $TD(\lambda)$ search can be much more efficient than MC search

TD Search

- Simulate episodes from the current (real) state s_t
- Estimate action-value function $Q(s, a)$
- For each step of simulation, update action-values by Sarsa

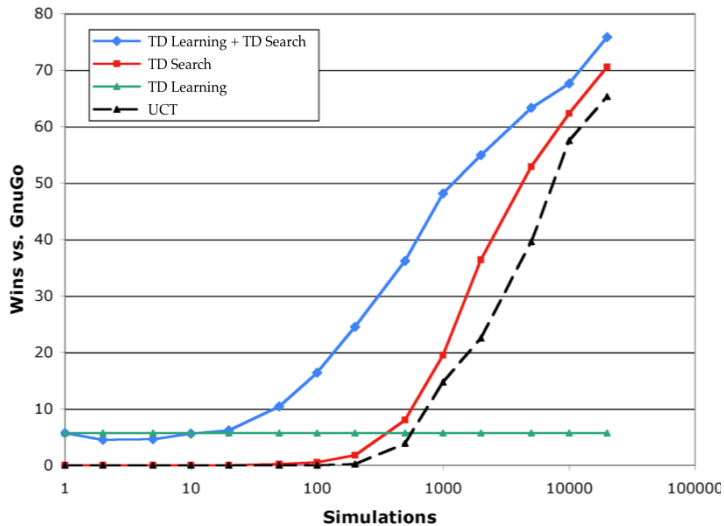
$$\Delta Q(S, A) = \alpha (R + \gamma Q(S', A') - Q(S, A))$$

- Select actions based on action-values $Q(s, a)$
e.g. ϵ -greedy
- May also use function approximation for Q

Dyna-2

- In Dyna-2, the agent stores two sets of feature weights
 - **Long-term** memory
 - **Short-term** (working) memory
- Long-term memory is updated from **real experience** using TD learning
 - General domain knowledge that applies to any episode
- Short-term memory is updated from simulated experience using TD search
 - Specific local knowledge about the current situation
- Over value function is sum of long and short-term memories

Results of TD search in Go



Lecture Summary

- Using models in reinforcement learning
 - Model Learning
 - Model-based RL
 - Architectures for planning and learning
 - Simulation-based search
- How to deal with changes in the model

We are done with tabular methods!